Task 2, Part C: Attention Gradients by Hand

Setup:

- $S = \frac{QK^T}{\sqrt{d_k}}$ (unscaled-then-scaled scores, shape $T \times T$)
- $P = \text{softmax}(S)$ (row-wise softmax, shape $T \times T$)
- $A = PV$ (attention output, shape $T \times d_v$)

Let L be the scalar loss. In standard backpropagation, when we write $\frac{\partial A}{\partial V}$, we are typically solving for how the upstream gradient $\frac{\partial L}{\partial A}$ flows into V, which we denote as $dV = \frac{\partial L}{\partial V}$.

1. Derive $\partial A/\partial V$

We want to find how the gradient flows from A to V. We know $A = PV$. In index notation, the element at row i and column j is:

$$A_{ij} = \sum_k P_{ik} V_{kj}$$

Taking the partial derivative of A_{ij} with respect to a specific element V_{kj}:

$$\frac{\partial A_{ij}}{\partial V_{kj}} = P_{ik}$$

Applying the chain rule using the upstream gradient dA (where $dA_{ij} = \frac{\partial L}{\partial A_{ij}}$):

$$dV_{kj} = \frac{\partial L}{\partial V_{kj}} = \sum_i \frac{\partial L}{\partial A_{ij}} \frac{\partial A_{ij}}{\partial V_{kj}} = \sum_i dA_{ij} P_{ik}$$

Rearranging to match matrix multiplication dimensions (summing over i means taking the dot product of the k-th column of P and the j-th column of dA):

$$dV = P^T \cdot dA$$

Answer: $\frac{\partial A}{\partial V} \implies \nabla_V = P^T(\nabla_A)$

2. Softmax Jacobian

We want to find $\frac{\partial p_i}{\partial s_j}$ for the softmax function $p_i = \frac{e^{s_i}}{\sum_k e^{s_k}}$.
Let the denominator be $\Sigma = \sum_k e^{s_k}$.

Case 1: $i = j$ (Derivative with respect to its own logit)
Using the quotient rule:

$$\frac{\partial p_i}{\partial s_i} = \frac{\frac{\partial}{\partial s_i}(e^{s_i}) \cdot \Sigma - e^{s_i} \cdot \frac{\partial}{\partial s_i}(\Sigma)}{\Sigma^2}$$

$$\frac{\partial p_i}{\partial s_i} = \frac{e^{s_i}\Sigma - e^{s_i}e^{s_i}}{\Sigma^2} = \left(\frac{e^{s_i}}{\Sigma}\right) - \left(\frac{e^{s_i}}{\Sigma}\right)^2$$

$$\frac{\partial p_i}{\partial s_i} = p_i - p_i^2 = p_i(1 - p_i)$$

Case 2: $i \neq j$ (Derivative with respect to a different logit)
Using the quotient rule:

$$\frac{\partial p_i}{\partial s_j} = \frac{\frac{\partial}{\partial s_j}(e^{s_i}) \cdot \Sigma - e^{s_i} \cdot \frac{\partial}{\partial s_j}(\Sigma)}{\Sigma^2}$$

Since $i \neq j$, $\frac{\partial}{\partial s_j}(e^{s_i}) = 0$. The derivative of the sum Σ with respect to s_j is e^{s_j}.

$$\frac{\partial p_i}{\partial s_j} = \frac{0 - e^{s_i} e^{s_j}}{\Sigma^2} = -\left(\frac{e^{s_i}}{\Sigma}\right)\left(\frac{e^{s_j}}{\Sigma}\right)$$

$$\frac{\partial p_i}{\partial s_j} = -p_i p_j$$

Combining Cases:
Using the Kronecker delta δ_{ij} (which is 1 if $i = j$ and 0 otherwise), we can combine these into a single equation:

$$\frac{\partial p_i}{\partial s_j} = p_i(\delta_{ij} - p_j)$$

3. Main Result: Derive $\partial A/\partial Q$

We use the chain rule to flow the gradient dA backward through P, then S, to Q.

Step 3a: Gradient through P
From $A = PV$, the gradient dP (shape $T \times T$) is:

$$dP = dA \cdot V^T$$

Step 3b: Gradient through S
Using the Softmax Jacobian derived in step 2, we calculate dS_{ij}. The softmax is applied row-wise, so the gradient for a single element S_{ij} depends on the entire i-th row of P:

$$dS_{ij} = \sum_k dP_{ik} \frac{\partial P_{ik}}{\partial S_{ij}}$$

Substitute the Jacobian:

$$dS_{ij} = \sum_k dP_{ik} P_{ik}(\delta_{kj} - P_{ij})$$

Distribute:

$$dS_{ij} = dP_{ij} P_{ij}(1 - P_{ij}) + \sum_{k \neq j} dP_{ik} P_{ik}(0 - P_{ij})$$

$$dS_{ij} = dP_{ij}P_{ij} - \sum_k dP_{ik}P_{ik}P_{ij}$$

Factor out P_{ij}:

$$dS_{ij} = P_{ij}\left(dP_{ij} - \sum_k dP_{ik}P_{ik}\right)$$

Step 3c: Gradient through Q
We know $S = \frac{QK^T}{\sqrt{d_k}}$.
In index notation: $S_{ij} = \frac{1}{\sqrt{d_k}} \sum_m Q_{im}K_{jm}$
The derivative with respect to Q_{im} is $\frac{K_{jm}}{\sqrt{d_k}}$.
Applying the chain rule using dS:

$$dQ_{im} = \sum_j dS_{ij} \frac{K_{jm}}{\sqrt{d_k}}$$

In matrix form:

$$dQ = dS \cdot \frac{K}{\sqrt{d_k}}$$

Final Combined Answer:
Let $dP = dA \cdot V^T$.
Let $dS_{ij} = P_{ij}\,(dP_{ij} - \sum_k dP_{ik}P_{ik})$.
Then $\nabla_Q = dS \cdot \frac{K}{\sqrt{d_k}}$.

4. Interpretation

Why does the gradient through softmax become very small when logits have large magnitudes?

When the input logits (S) have very large magnitudes, the exponentiation in the softmax function causes the largest logit to completely dominate the sum. As a result, the output probability distribution P becomes highly peaked: the largest value approaches 1.0, and all other values approach 0.0.

Looking at the Softmax Jacobian we derived: $\frac{\partial p_i}{\partial s_j} = p_i(\delta_{ij} - p_j)$.

- If $p_i \approx 1$, then $(1 - p_i) \approx 0$, so the diagonal gradient $p_i(1 - p_i) \to 0$.
- If $p_i \approx 0$, then p_i multiplied by anything is 0, so both the diagonal and off-diagonal gradients $(-p_i p_j) \to 0$.

Therefore, when logits are large, every entry in the Jacobian matrix approaches zero. The gradient vanishes, and the network stops learning.

Connection to $\sqrt{d_k}$:
The pre-softmax logits are calculated via a dot product QK^T. If Q and K vectors have a variance of 1, their dot product across d_k dimensions will have a variance of d_k. For large embedding dimensions, this causes naturally massive logits. Dividing by $\sqrt{d_k}$ standardizes the variance

back to 1, keeping the logits small and ensuring the softmax operates in its ”soft,” differentiable region where gradients can flow.